

■ Python Decorators — Complete Guide + Interview Q&A; From Basics to Advanced with Real-world Examples

🧠 1 What is a Decorator?

Definition (Basic):

A **decorator** in Python is a function that **adds extra functionality** to another function **without modifying its code directly**.

Simpler meaning:

It's like giving a function a **special costume** — it behaves the same but with **extra powers**.

Technical definition (Intermediate):

A **decorator** is a **higher-order function** that takes another function as an argument, wraps some extra code around it, and returns a new function.

⚙️ 2 Why Use Decorators?

- ✅ To **add reusable functionality** (like logging, timing, validation)
 - ✅ To follow the **DRY (Don't Repeat Yourself)** principle
 - ✅ To make code **clean and maintainable**
 - ✅ To handle **cross-cutting concerns** like authentication, caching, etc.
-

📖 3 How Decorators Work (Step-by-Step)

Example 1 — Without Using @ Symbol:

```
def greet():  
    print("Hello!")
```

```
def add_message(func):
```

```
def wrapper():  
    print("Before greeting")  
    func()  
    print("After greeting")  
return wrapper
```

```
new_func = add_message(greet)  
new_func()
```

Output:

Before greeting

Hello!

After greeting

Example 2 — Using @decorator_name Syntax:

```
def add_message(func):  
    def wrapper():  
        print("Before greeting")  
        func()  
        print("After greeting")  
    return wrapper
```

```
@add_message
```

```
def greet():  
    print("Hello!")
```

```
greet()
```

Same output — cleaner syntax.

💡 4 Decorators with Arguments

Example:

```
def check_ints(func):  
    def wrapper(a, b):  
        if not isinstance(a, int) or not isinstance(b, int):  
            return "Invalid input!"  
        return func(a, b)  
    return wrapper
```

```
@check_ints
```

```
def add(a, b):  
    return a + b
```

```
print(add(2, 3))
```

```
print(add("2", 3))
```

🔒 5 Decorators with *args and **kwargs

To handle **any number of arguments**:

```
def logger(func):  
    def wrapper(*args, **kwargs):  
        print(f'Function '{func.__name__}' called with {args}, {kwargs}')  
        return func(*args, **kwargs)  
    return wrapper
```

```
@logger
```

```
def multiply(x, y):  
    return x * y
```

```
print(multiply(5, 6))
```

6 Returning Values from Decorated Functions

Always ensure the wrapper returns the result:

```
def double_result(func):  
    def wrapper(*args, **kwargs):  
        result = func(*args, **kwargs)  
        return result * 2  
    return wrapper
```

```
@double_result  
def get_number():  
    return 10
```

```
print(get_number()) # Output: 20
```



When you use decorators, they replace the original function's metadata.

To preserve it, use `@functools.wraps`.

```
from functools import wraps
```

```
def smart_decorator(func):  
    @wraps(func)  
    def wrapper(*args, **kwargs):  
        """Wrapper function"""  
        print("Smart decorator active!")
```

```

        return func(*args, **kwargs)

    return wrapper

@smart_decorator
def say_hi():
    """Say hi function"""
    print("Hi!")

say_hi()

print(say_hi.__name__) # say_hi
print(say_hi.__doc__) # Say hi function

```



8 Decorators with Parameters (Decorator Factory)

You can also create decorators that **accept arguments**.

```

def repeat(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(n):
                func(*args, **kwargs)
            return wrapper
        return decorator
    return decorator

```

```

@repeat(3)
def greet():
    print("Hello!")

```

```

greet()

```

9 Class-Based Decorators

You can also use **classes** as decorators if they implement `__call__`.

```
class CountCalls:
```

```
    def __init__(self, func):
```

```
        self.func = func
```

```
        self.count = 0
```

```
    def __call__(self, *args, **kwargs):
```

```
        self.count += 1
```

```
        print(f'{self.func.__name__} has been called {self.count} times.')
```

```
        return self.func(*args, **kwargs)
```

```
@CountCalls
```

```
def say_hello():
```

```
    print("Hello!")
```

```
say_hello()
```

```
say_hello()
```

10 Real-World Examples

 **Logging:**

```
import datetime
```

```
def log_time(func):
```

```
    def wrapper(*args, **kwargs):
```

```
    print(f"Running '{func.__name__}' at {datetime.datetime.now()}")
    return func(*args, **kwargs)

return wrapper
```

@log_time

```
def process_data():
    print("Processing data...")
```

process_data()

Authentication:

```
def requires_login(func):
    def wrapper(user):
        if not user.get("is_logged_in"):
            return "Access Denied"

        return func(user)

    return wrapper
```

@requires_login

```
def profile(user):
    return f"Welcome {user['name']}"
```

```
print(profile({"name": "Shanmukha", "is_logged_in": True}))
print(profile({"name": "Guest", "is_logged_in": False}))
```

MY Decorators Practice

(Decorators are higher-order functions)

1. Basic Decorator Example

```
def check_msg(func):
    def wrapper():
        print("Before the original function")
        func()
        print("After the original function")
    return wrapper

@check_msg
def say_hello():
    print("Hello, World!")

@check_msg
def good_evening():
    print("Good Evening!")

good_evening()
```

2. Decorator to Check Integer Inputs

```
def check_int(func):
    def wrapper(a, b):
        print(type(a), type(b))
        if not isinstance(a, int) or not isinstance(b, int):
            return "Invalid inputs — please provide integers only."
        return func(a, b)
    return wrapper

@check_int
def add(a, b):
    return a + b
```



```
print(add(77, 2))
```

3. Decorator to Check String Input

```
def check_str(func):  
    def wrapper(name):  
        if not isinstance(name, str):  
            return "Invalid name — must be a string."  
        return func(name)  
    return wrapper  
  
@check_str  
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet(1233))
```

4. Decorator to Block Function When Zero is in Arguments

```
def has_zero(func):  
    def wrapper(*args):  
        if 0 in args:  
            return func("Zero found! Function skipped.")  
        return func(*args)  
    return wrapper  
  
@has_zero  
def numbers(*args):  
    return args  
  
print(numbers(1, 2, 3, 0, 4, 5))
```

5. Decorator to Filter Specific Names

```
def hate_names(func):  
    def wrapper(*args):
```

```
    if 'babar' in args:
        return func("Please remove 'babar' — I don't like them ")
    return func(*args)
return wrapper

@hate_names
def cricket_players(*args):
    return args

print(cricket_players('virat', 'dhoni', 'babar', 'shannu'))
```